

Universidad Tecnológica de Santiago

"Tarea 5 – Actividad integradora: app que consume un servicio web"

Nombre:
César Miguel Moronta Guillen

Matricula:
2-22-0299

Carrera:
Ingeniería en Sistemas Computacionales

Asignatura:
Programación de Dispositivos Móviles

Profesor:
Joan Miguel Tejada Caba

25 de Octubre del 2026

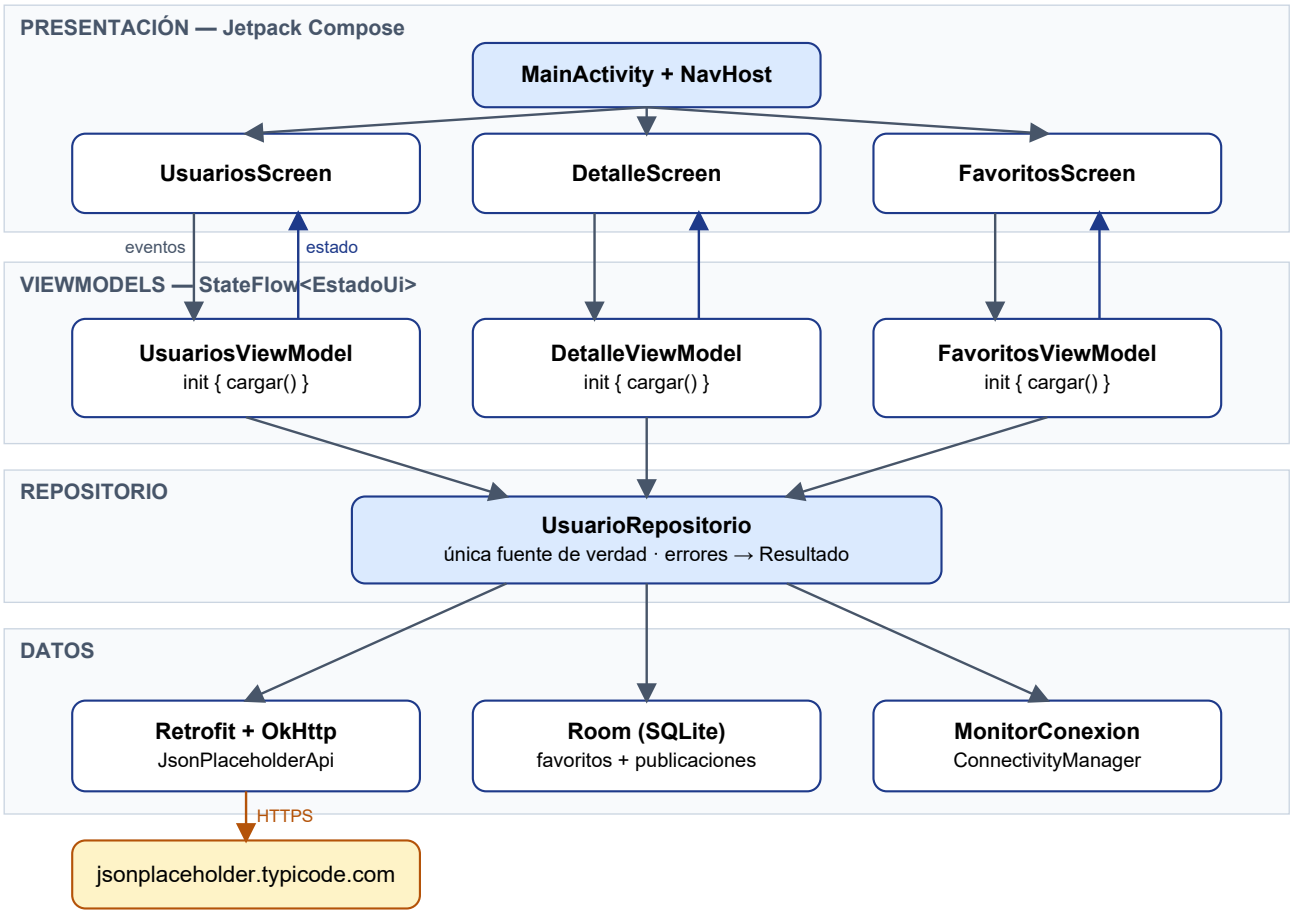
Santiago, Republica Dominicana

1. Descripción de la app

Directorio es una app Android que consume la API pública **JSONPlaceholder**. Muestra la lista de usuarios, el detalle de cada uno con sus publicaciones, y permite marcar favoritos que se guardan en el teléfono con Room para consultarlos sin conexión. La interfaz informa siempre en qué estado está (cargando, éxito, error o sin Internet) y nunca se cierra por un fallo de red.

Requisito	Implementación
1. Lista de usuarios	GET /users → nombre, correo y company.name (UsuariosScreen)
2. Detalle y navegación	Al tocar un usuario: datos completos + GET /posts?userId={id} (DetalleScreen, Navigation Compose)
3. Favoritos persistentes	Room: tablas favoritos y publicaciones. Se ven sin conexión (FavoritosScreen)
4. Estados de la interfaz	EstadoUi: Cargando, Exito, Error, SinConexion + botón Reintentar; reintento automático al volver Internet
5. Librerías y permisos	Retrofit 2.11 + Gson + OkHttp. Permisos INTERNET y ACCESS_NETWORK_STATE en el AndroidManifest
6. Arquitectura	MVVM por capas (sección 2) y relación con el ciclo de vida (sección 3)

2. Arquitectura: MVVM por capas



- **Presentación (Compose):** las pantallas solo dibujan el estado y envían eventos (tocar, reintentar, marcar favorito). No conocen Retrofit ni Room.
- **ViewModel:** mantiene el estado de cada pantalla en un `StateFlow<EstadoUi>` y sobrevive a la rotación. Lanza las peticiones en `viewModelScope`.
- **Repositorio:** única fuente de verdad. Decide si los datos vienen de la API o de Room y convierte toda excepción en un `Resultado` (Exito, SinConexion o Error).
- **Datos:** Retrofit (remoto), Room (local) y MonitorConexion (estado de la red con ConnectivityManager).
- **Dependencias:** se crean una sola vez en la clase Application (`DirectorioApp`) con un contenedor manual.

Paquetes

```
com.cmoronta.directorio
├── DirectorioApp.kt          Application + contenedor de dependencias
├── MainActivity.kt           única Activity, NavHost (usuarios, detalle/{id}, favoritos)
├── data/modelo               Usuario, Publicacion, Resultado
├── data/remoto               JsonPlaceholderApi (Retrofit), DTOs, ClienteRetrofit
├── data/local                Room: FavoritoEntity, PublicacionEntity, FavoritoDao, BaseDatos
├── data/red                  MonitorConexion
├── data/repositorio          UsuarioRepositorio
└── ui/                       comun (estados), usuarios, detalle, favoritos, tema
```

3. Ciclo de vida y carga de datos

La carga de datos se relaciona con el ciclo de vida de Android de la siguiente forma:

Momento	Qué ocurre con los datos
Application.onCreate()	Se crean Retrofit, Room y el repositorio una sola vez (DirectorioApp).
Activity.onCreate() → setContent	Se dibuja la navegación; cada pantalla obtiene su ViewModel con viewModel(factory = …).
Creación del ViewModel (init {})	Se lanza cargar() en viewModelScope. La pantalla muestra el estado Cargando.
onStart … onStop	collectAsStateWithLifecycle() observa el StateFlow solo mientras la pantalla está visible; en onStop deja de observar.
onResume	LifecycleEventEffect(ON_RESUME): si la última carga falló (error o sin conexión), se reintenta al volver a la app.
Rotación / cambio de configuración	La Activity se recrea pero el ViewModel sobrevive: no se repite la descarga.
Muerte del proceso	El id del detalle se recupera de SavedStateHandle (argumento de navegación detalle/{id}).
ViewModel.onCleared()	Al salir de la pantalla se cancela viewModelScope y cualquier petición en curso.

Además, si la lista está en estado **SinConexion** y vuelve Internet, MonitorConexion lo detecta y el ViewModel recarga automáticamente.

Fragmento: carga en init {} y reintento en ON_RESUME

```
init {
    // Se ejecuta UNA vez al crear el ViewModel (primer onCreate de la pantalla).
    // Al rotar el teléfono el ViewModel sobrevive, así que NO se vuelve a descargar.
    cargar()

    // Si estábamos sin conexión y vuelve Internet, reintenta solo.
    viewModelScope.launch {
        conexion.hayConexion.collect { online ->
            if (online && carga.value is EstadoUi.SinConexion) cargar()
        }
    }
}

fun cargar() {
    viewModelScope.launch {
        carga.value = EstadoUi.Cargando
        carga.value = repositorio.obtenerUsuarios().aEstadoUi()
    }
}

/** Llamado en ON_RESUME: si la última carga falló, se vuelve a intentar al regresar a la app. */
fun alReanudar() {
    if (carga.value is EstadoUi.Error || carga.value is EstadoUi.SinConexion) cargar()
}

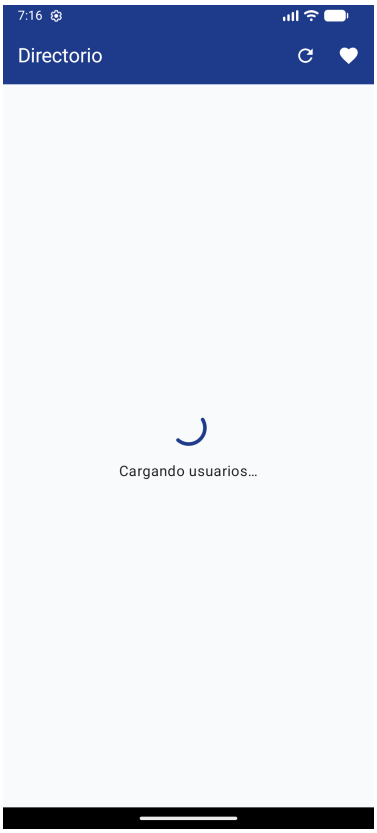
// collectAsStateWithLifecycle: solo observa mientras la pantalla está visible (entre onStart y onStop).
val estado by viewModel.estado.collectAsStateWithLifecycle()
val hayConexion by viewModel.hayConexion.collectAsStateWithLifecycle()

// ON_RESUME: al volver a la app (p. ej. tras activar el Wi-Fi), reintenta si había fallado.
```

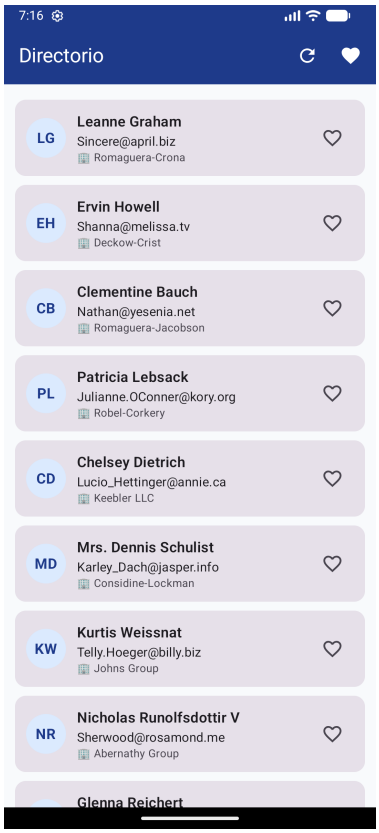
```
LifecycleEventEffect(Lifecycle.Event.ON_RESUME) { viewModel.alReanudar() }
```

4. Capturas

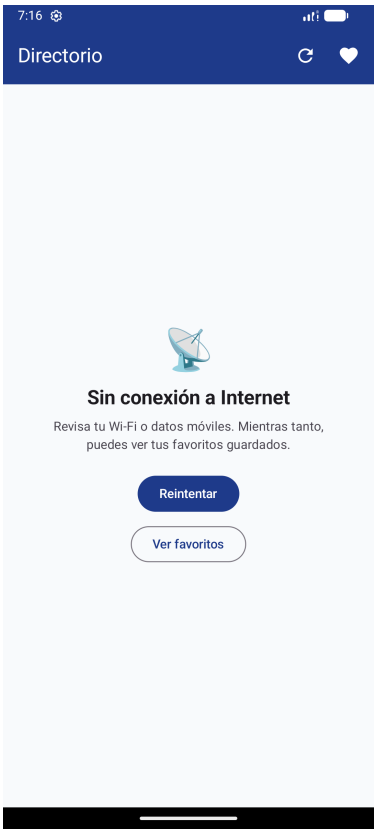
Los tres estados de la interfaz



Carga

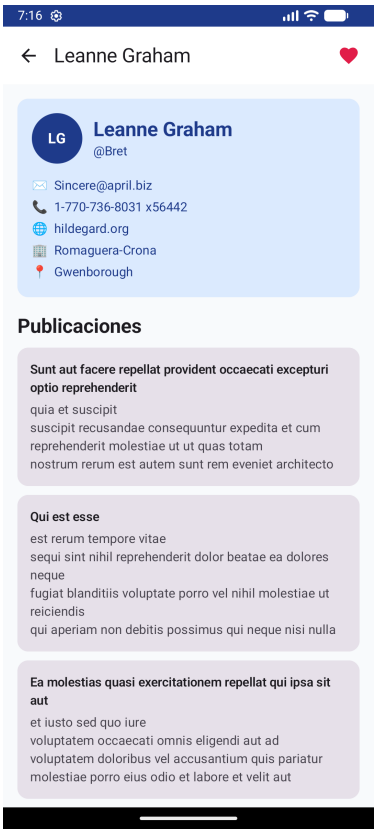


Éxito: lista de usuarios

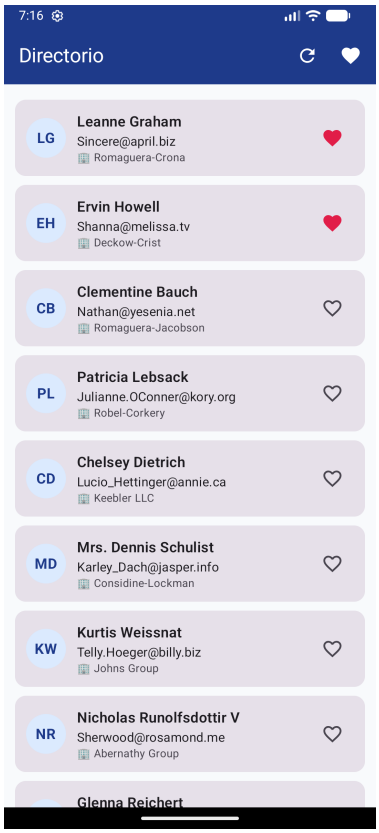


Error / sin conexión

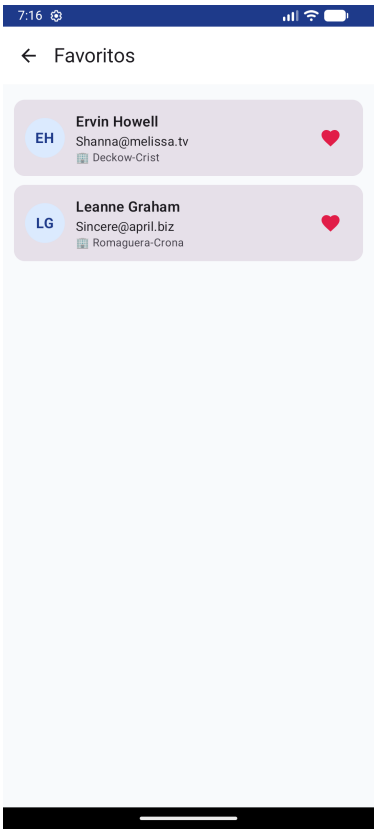
Detalle y favoritos



Detalle con publicaciones

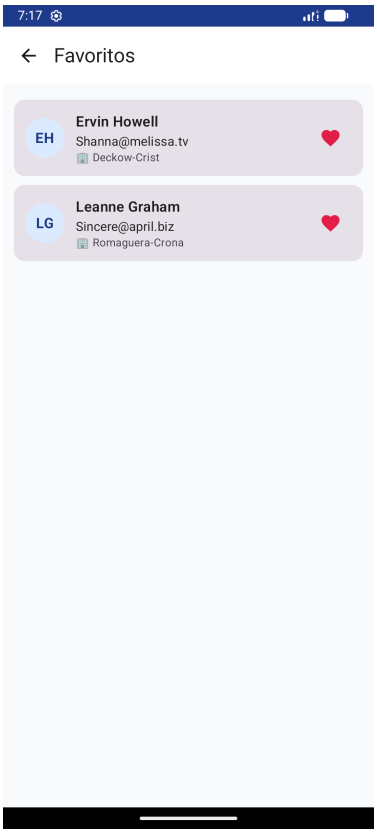


Favoritos marcados

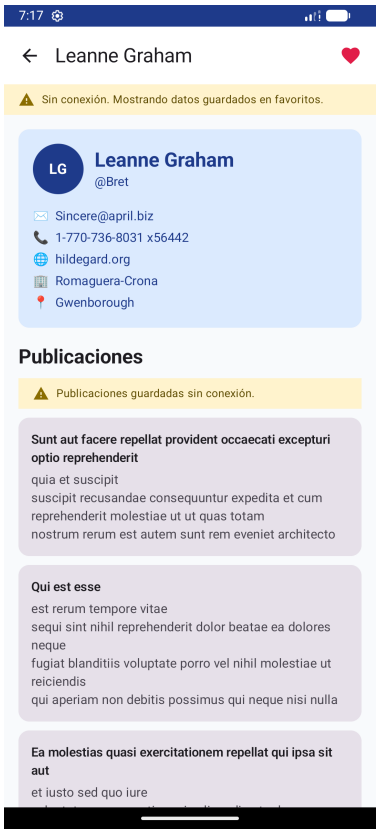


Pantalla de favoritos

Favoritos sin conexión (datos desde Room)



Favoritos sin Internet



Detalle y publicaciones sin Internet

5. Código del consumo de la API (Retrofit)

Permisos en AndroidManifest.xml

```
<!-- Necesario para consumir la API de JSONPlaceholder -->
<uses-permission android:name="android.permission.INTERNET" />
<!-- Para detectar si hay conexión y mostrar el mensaje "Sin Internet" -->
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

Interfaz Retrofit y DTOs (data/remoto/JsonPlaceholderApi.kt)

```
interface JsonPlaceholderApi {

    @GET("users")
    suspend fun obtenerUsuarios(): List<UsuarioDto>

    @GET("users/{id}")
    suspend fun obtenerUsuario(@Path("id") id: Int): UsuarioDto

    @GET("posts")
    suspend fun obtenerPublicaciones(@Query("userId") usuarioId: Int): List<PublicacionDto>

    companion object {
        const val URL_BASE = "https://jsonplaceholder.typicode.com/"
    }
}
```

Cliente Retrofit (data/remoto/ClienteRetrofit.kt)

```
object ClienteRetrofit {

    fun crearApi(): JsonPlaceholderApi {
        val registro = HttpLoggingInterceptor().apply {
            level = if (BuildConfig.DEBUG) HttpLoggingInterceptor.Level.BASIC else HttpLoggingInterceptor.Level.NONE
        }
        val http = OkHttpClient.Builder()
            .connectTimeout(10, TimeUnit.SECONDS)
            .readTimeout(15, TimeUnit.SECONDS)
            .addInterceptor(registro)
            .build()

        return Retrofit.Builder()
            .baseUrl(JsonPlaceholderApi.URL_BASE)
            .client(http)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(JsonPlaceholderApi::class.java)
    }
}
```

Manejo de errores en el repositorio: la app nunca se cierra

```
suspend fun obtenerUsuarios(): Resultado<List<Usuario>> = llamar {
    api.obtenerUsuarios().map { it.aModelo() }.also { cacheUsuarios = it }
}

    /** Ejecuta una llamada a la API y traduce los errores a mensajes claros. */
    private suspend fun <T> llamar(bloque: suspend () -> T): Resultado<T> = try {
        Resultado.Exito(bloque())
    } catch (e: CancellationException) {
        throw e // la corrutina se canceló (p. ej. se cerró la pantalla): no es un error
    } catch (e: IOException) {
        if (!conexion.hayConexionAhora()) Resultado.SinConexion
        else Resultado.Error("No se pudo contactar al servidor. Revisa tu conexión e inténtalo de nuevo.")
    } catch (e: HttpException) {
        Resultado.Error("El servidor respondió con un error (${e.code()}).")
    } catch (e: Exception) {
        Resultado.Error("Ocurrió un error inesperado: ${e.message ?: e.javaClass.simpleName}")
```

```
    }
}
```

Estados de la interfaz

```
/** Los estados posibles de una pantalla que carga datos. */
sealed interface EstadoUi<out T> {
    data object Cargando : EstadoUi<Nothing>
    data class Exito<T>(val datos: T, val desdeCache: Boolean = false) : EstadoUi<T>
    data object SinConexion : EstadoUi<Nothing>
    data class Error(val mensaje: String) : EstadoUi<Nothing>
}

when (val e = estado) {
    EstadoUi.Cargando -> PantallaCargando("Cargando usuarios…")
    EstadoUi.SinConexion -> PantallaSinConexion(alReintentar = viewModel::cargar, alVerFavoritos = alAbrirFavoritos)
    is EstadoUi.Error -> PantallaError(e.mensaje, alReintentar = viewModel::cargar)
    is EstadoUi.Exito ->
        if (e.datos.isEmpty()) PantallaMensaje("  ", "Sin usuarios", "La API no devolvió usuarios.")
        else ListaUsuarios(e.datos, alAbrirUsuario, viewModel::alternarFavorito)
}
```

6. Código de la persistencia (Room)

Entidades, DAO y base de datos (data/local/BaseDatos.kt)

```
/** Usuario marcado como favorito, con todos sus datos para poder verlo sin conexión. */
@Entity(tableName = "favoritos")
data class FavoritoEntity(
    @PrimaryKey val id: Int,
    val nombre: String,
    val usuario: String,
    val correo: String,
    val telefono: String,
    val web: String,
    val empresa: String,
    val ciudad: String,
    val guardadoEn: Long = System.currentTimeMillis(),
)

/** Publicaciones de los favoritos, guardadas para mostrarlas sin conexión. */
@Entity(tableName = "publicaciones", indices = [Index("usuarioId")])
data class PublicacionEntity(
    @PrimaryKey val id: Int,
    val usuarioId: Int,
    val titulo: String,
    val cuerpo: String,
)

@Dao
interface FavoritoDao {

    @Query("SELECT * FROM favoritos ORDER BY nombre")
    fun observarFavoritos(): Flow<List<FavoritoEntity>>

    @Query("SELECT id FROM favoritos")
    fun observarIds(): Flow<List<Int>>

    @Query("SELECT EXISTS(SELECT 1 FROM favoritos WHERE id = :id)")
    fun observarEsFavorito(id: Int): Flow<Boolean>

    @Query("SELECT * FROM favoritos WHERE id = :id")
    suspend fun obtener(id: Int): FavoritoEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertar(favorito: FavoritoEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertarPublicaciones(publicaciones: List<PublicacionEntity>)

    @Query("SELECT * FROM publicaciones WHERE usuarioId = :usuarioId ORDER BY id")
    suspend fun publicacionesDe(usuarioId: Int): List<PublicacionEntity>

    @Query("DELETE FROM favoritos WHERE id = :id")
    suspend fun eliminarFavorito(id: Int)

    @Query("DELETE FROM publicaciones WHERE usuarioId = :usuarioId")
    suspend fun eliminarPublicaciones(usuarioId: Int)

    @Transaction
    suspend fun guardar(favorito: FavoritoEntity, publicaciones: List<PublicacionEntity>) {
        insertar(favorito)
        if (publicaciones.isNotEmpty()) insertarPublicaciones(publicaciones)
    }

    @Transaction
    suspend fun quitar(id: Int) {
        eliminarPublicaciones(id)
        eliminarFavorito(id)
    }
}
```

```
@Database(entities = [FavoritoEntity::class, PublicacionEntity::class], version = 1, exportSchema = false)
abstract class BaseDatos : RoomDatabase() {
    abstract fun favoritoDao(): FavoritoDao

    companion object {
        fun crear(contexto: Context): BaseDatos =
            Room.databaseBuilder(contexto, BaseDatos::class.java, "directorio.db").build()
    }
}
```

Favoritos y publicaciones sin conexión (repositorio)

```
/** Publicaciones del usuario. Si es favorito se guardan en Room y se usan cuando no hay conexión. */
suspend fun obtenerPublicaciones(usuarioId: Int): Resultado<List<Publicacion>> {
    val remoto = llamar { api.obtenerPublicaciones(usuarioId).map { it.aModelo() } }
    if (remoto is Resultado.Exito) {
        if (dao.obtener(usuarioId) != null) dao.insertarPublicaciones(remoto.datos.map { it.aEntidad() })
        return remoto
    }
    val guardadas = dao.publicacionesDe(usuarioId)
    return if (guardadas.isNotEmpty()) Resultado.Exito(guardadas.map { it.aModelo() }, desdeCache = true) else remoto
}

suspend fun guardarFavorito(usuario: Usuario, publicaciones: List<Publicacion> = emptyList()) =
    dao.guardar(usuario.aEntidad(), publicaciones.map { it.aEntidad() })

suspend fun quitarFavorito(id: Int) = dao.quitar(id)
```

7. Pruebas

Pruebas unitarias del repositorio con dobles de prueba (API, DAO y conexión falsos). Resultado: **5 de 5 pruebas superadas**.

Prueba	Qué verifica
lista de usuarios exitosa	Nombre, correo y company.name se mapean correctamente
sin internet devuelve SinConexion	Una IOException sin red no cierra la app: devuelve el estado SinConexion
error del servidor	Un HTTP 500 se convierte en un mensaje claro para el usuario
favorito sin conexión	Tras guardar un favorito y perder Internet, el usuario y sus publicaciones se leen de Room
quitar favorito	Al quitarlo se borran también sus publicaciones guardadas

8. Conclusiones

- Separar la app en capas (UI, ViewModel, repositorio y datos) permitió cambiar el origen de los datos (API o Room) sin tocar las pantallas, y probar el repositorio sin un teléfono ni conexión real.
- Modelar los estados de la interfaz como un tipo sellado (EstadoUi) obliga a contemplar la carga, el error y la falta de Internet en cada pantalla; el usuario siempre recibe un mensaje claro y un botón para reintentar.
- Capturar las excepciones en un solo punto del repositorio (IOException, HttpException y otras) evita que la app se cierre por fallos de red o del servidor.
- El ViewModel resuelve el problema del ciclo de vida: los datos se cargan en init {}, sobreviven a la rotación, y collectAsStateWithLifecycle solo observa mientras la pantalla está visible.
- Guardar en Room no solo el favorito sino también sus publicaciones hace que la experiencia sin conexión sea completa.

9. Enlace al repositorio

(pegar aquí el enlace al repositorio)